PUC Minas

Testes de Software

Refatoração de Testes e Detecção de Test Smells

Aluno: Gabriel Reis Lebron de Oliveira
Matrícula: 855324
Repositório: https://github.com/GabrielReiiss/test-smelly

Belo Horizonte, outubro de 2026

1. Análise de Smells

A suíte original `userService.smelly.test.js` possui cinco testes para o `UserService`. Todos passam, quatro são executados e um é ignorado com `test.skip`. Mesmo assim, a leitura manual dos testes revelou vários Test Smells.

1.1 Lógica Condicional

Onde aparece: nos testes “deve desativar usuários se eles não forem administradores” e “deve falhar ao criar usuário menor de idade”.

Por que é um mau cheiro: o primeiro teste percorre uma lista com `for` e usa `if/else` para decidir quais `expect` executar. O segundo coloca a única asserção dentro de um `catch`: `try { createUser('Menor', ..., 17) } catch (e) { expect(e.message)... }`. Testes devem ser determinísticos e lineares, quem lê o teste deveria ver exatamente o que é verificado, sem precisar simular a execução de cabeça.

Risco: as asserções podem simplesmente não ser executadas, e o teste passa sem testar nada. No caso do `try/catch`, se a validação de idade for removida do código, nenhuma exceção é lançada, o `expect` nunca roda e o teste continua passando, escondendo o bug.

1.2 Teste Ansioso

Onde aparece: no teste “deve criar e buscar um usuário corretamente”.

Por que é um mau cheiro: um único teste verifica dois comportamentos de métodos diferentes (`createUser` e `getUserById`), com duas ações. Os próprios comentários marcam “Act 1” e “Act 2”, o que quebra o padrão AAA. As três asserções sem descrição também caracterizam uma Roleta de Asserções.

Risco: quando o teste falha, não se sabe qual comportamento quebrou, e uma falha na criação impede que a busca seja verificada.

1.3 Teste Frágil

Onde aparece: no teste “deve gerar um relatório de usuários formatado”, que compara o relatório com a linha exata `` `ID: ${usuario1.id}, Nome: Alice, Status: ativo\n` `` e com o cabeçalho `'--- Relatório de Usuários ---'`.

Por que é um mau cheiro: o teste está acoplado ao formato exato da string (ordem dos campos, separadores, quebra de linha e cabeçalho), ou seja, à implementação. Ele deveria verificar o comportamento, o relatório lista os usuários e os seus status. O próprio `userService.js` avisa que “o formato do relatório pode mudar no futuro”.

Risco: qualquer mudança cosmética quebra o teste, mesmo com o relatório correto. Esses falsos alarmes fazem a equipe desconfiar da suíte e gastar tempo corrigindo testes em vez de código.

Também foram observados um `test.skip` vazio e o método `_clearDB()`, que existe no código de produção só para os testes.

2. Processo de Refatoração

O teste escolhido foi “deve desativar usuários se eles não forem administradores”, o mais problemático da suíte, concentra três dos quatro erros do ESLint e mistura dois cenários opostos em um único teste.

A refatoração seguiu quatro decisões, aplicadas no código abaixo:

- **Um cenário, um teste.** Cada um dos dois cenários opostos virou um teste próprio. Sem for e if/else, todo expect passa a ser sempre executado, o que elimina a Lógica Condicional.
- **AAA explícito.** O *Arrange* cria só o usuário necessário, o *Act* é uma única chamada a `deactivateUser` e o *Assert* verifica o retorno e o novo estado.
- **Nomes descritivos.** Cada nome descreve uma regra de negócio, e uma falha já mostra qual regra quebrou.
- **Asserção mais forte.** Para o admin, o teste também confirma que o status continua `'ativo'`, e não só o retorno `false`.

Antes: userService.smelly.test.js

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  // O teste tem um loop e um if, tornando-o complexo e menos claro.
  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      // Este expect só roda para o usuário comum.
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      // E este só roda para o admin.
      expect(resultado).toBe(false);
    }
  }
});
```

Depois: userService.clean.test.js

```
test('deve desativar um usuário comum', () => {
  // Arrange
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);

  // Act
  const resultado = userService.deactivateUser(usuarioComum.id);

  // Assert
  expect(resultado).toBe(true);
  expect(userService.getUserById(usuarioComum.id).status).toBe('inativo');
});

test('não deve desativar um usuário administrador', () => {
  // Arrange
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  // Act
  const resultado = userService.deactivateUser(usuarioAdmin.id);

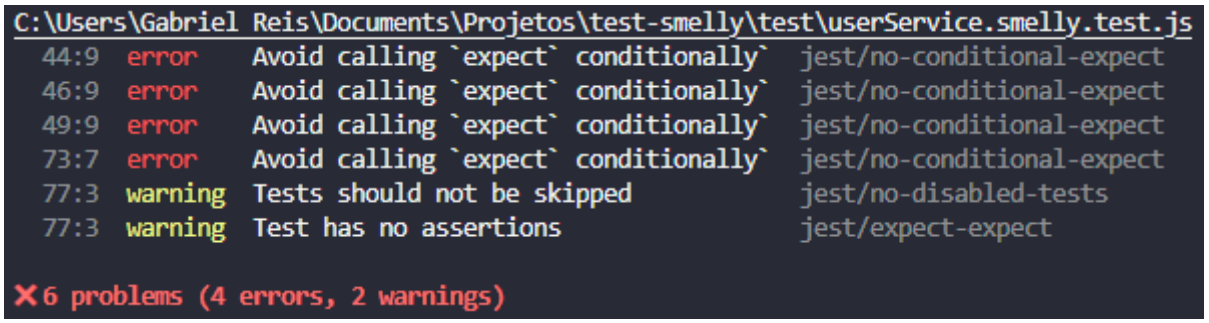
  // Assert
  expect(resultado).toBe(false);
  expect(userService.getUserById(usuarioAdmin.id).status).toBe('ativo');
});
```

Os demais smells foram corrigidos usando o mesmo raciocínio, e a suíte limpa passou a ter 8 testes focados, agrupados por método:

Smell	Antes (smelly)	Depois (clean)
Teste Ansioso	Criar e buscar no mesmo teste	3 testes, criar com os dados corretos, gerar um id e buscar pelo id
Lógica Condicional	`expect` dentro do `catch`	`expect(criarMenor).toThrow(...)`, que falha se nenhuma exceção for lançada
Teste Frágil	Compara a linha exata e o cabeçalho	Verifica o conteúdo com `toContain('Alice')`, `toContain('Bob')` e `toContain('ativo')`
Teste desativado	`test.skip` vazio	Teste implementado, o relatório informa “Nenhum usuário cadastrado”

3. Relatório da Ferramenta

O ESLint foi instalado com o `eslint-plugin-jest` e configurado com o `.eslintrc.json` do enunciado. Como a versão mais recente (ESLint 10) não aceita mais o formato `.eslintrc`, a dependência foi fixada na versão 8.57.1.



A ferramenta encontrou 6 problemas, todos no arquivo *smelly*, e indicou a linha exata e a regra violada em cada um:

Regra do ESLint	Linhas	Smell correspondente (análise manual)
`jest/no-conditional-expect` (erro)	44, 46, 49	Lógica Condicional: `for` + `if/else`
`jest/no-conditional-expect` (erro)	73	Lógica Condicional: `expect` dentro do `catch`
`jest/no-disabled-tests` e `jest/expect-expect` (avisos)	77	Teste desativado (`test.skip`) e sem asserções

A regra `no-conditional-expect` detectou automaticamente o smell mais perigoso da suíte, o `expect` que pode não ser executado, nos mesmos dois testes da análise manual. Já o Teste Ansioso, o Teste Frágil e o *código de produção criado só para testes* não foram detectados, pois dependem da intenção do teste, e não de um padrão no código. A ferramenta complementa a revisão manual, mas não a substitui.

Após a refatoração, o ESLint não aponta nenhum erro ou aviso, e o `npm test` passa nas duas suítes, 12 testes aprovados e 1 ignorado.

4. Conclusão

Este trabalho mostrou que um teste passar não significa que ele seja bom. Todos os testes da suíte original passavam, mas alguns nunca conseguiriam falhar e outros quebrariam por uma mudança irrelevante de formatação.

Escrever testes limpos, com o padrão AAA, um cenário por teste e nomes que descrevem regras de negócio, transforma a suíte em uma documentação viva do sistema, ela fica mais fácil de manter e só falha quando o comportamento realmente muda. Ferramentas de análise estática como o ESLint automatizam parte dessa vigilância, detectando em segundos padrões perigosos, e podem rodar a cada commit para impedir que novos smells entrem no projeto. Como alguns smells dependem de interpretação, é a combinação da ferramenta com a revisão humana que garante uma suíte confiável e um projeto sustentável.